When these trees are used to store huge collections of large items, these properties allow to use memory paging efficiently: by having lightweight nodes, it is more likely the whole tree will fit in memory, while items can be stored on disk, and leaves can be loaded on a need-to-basis; since it is also likely that after accessing an item `X` applications will shortly after need to access one of its contiguous items, by loading in memory the whole B-tree leaf containing `X` we can make sure to reduce the disk reads as much as possible.

Not surprisingly, exactly for these reasons B-trees have been the core of many SQL database engines since their invention[5] - and even today they are still the data structure of choice for storing indices.

10.2.2 From B-Tree to R-tree

R-trees extends the main ideas behind B+ trees to the multidimensional case: while for unidimensional data each node corresponds to an interval (the range from the left-most and right-most keys in its sub-tree, which are in turn its minimum and maximum keys), in R-trees each node `N` covers a rectangle (or better a hyper-rectangle in the most generic case) whose corners are defined by the minimum and maximum of each coordinate, over all the points in the subtree rooted at `N`.

Similarly to B-trees, R-trees are also parametric: instead of a branching factor `d` controlling the minimum number of entries per node, R-trees requires their client to provide two parameters on creation: